

WEEK – 1, 2

Fundamentals of Object Oriented Programming: Object oriented paradigm, Object and Classes, Data Abstraction and Encapsulation, Inheritance, Polymorphism, Dynamic Binding.

1. Sample Case study on class & objects CO1

Bank Account Management System:

Problem Statement: Design a system to manage bank accounts.

Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

Object Usage: Create objects for each customer's account and perform transactions like deposits, withdrawals, and transfers.

Algorithm for the code:

2. Algorithm

Algorithm: Bank Account Management System

Step 1: Start.

Step 2: Create a SavingsAccount object with:

- **Account number = 100**
- **Balance = 5000**
- **Interest rate = 5%**

Step 3: Create a CurrentAccount object with:

- **Account number = 101**
- **Balance = 3000**
- **Overdraft limit = 2000**

Step 4: Display the following menu:

- 1. Deposit (Savings)**
- 2. Withdraw (Savings)**
- 3. Interest (Savings)**
- 4. Withdraw (Current)**
- 5. Transfer Savings -> Current**
- 6. Show Accounts**

7. Exit

Step 5: Read the user's choice.

Step 6: If choice is 1:

- **Read deposit amount.**
- **Add amount to Savings Account balance.**
- **Display updated balance.**

Step 7: If choice is 2:

- **Read withdrawal amount.**
- **Check whether amount is less than or equal to the Savings balance.**
- **If yes, withdraw the amount.**
- **Otherwise, display "insufficient balance".**

Step 8: If choice is 3:

- **Calculate interest using:**

$\text{Interest} = \text{Balance} \times \text{Rate} / 100$

- **Display the interest.**

Step 9: If choice is 4:

- **Read withdrawal amount.**
- **Check whether:**

$\text{Amount} \leq \text{Balance} + \text{Overdraft Limit}$

- **If yes, withdraw the amount.**
- **Otherwise, display "overdraft limit crossed".**

Step 10: If choice is 5:

- **Read transfer amount.**
- **Check whether Savings balance is sufficient.**
- **If sufficient:**
 - **Deduct amount from Savings.**
 - **Add amount to Current Account.**
- **Otherwise, display transfer failure.**

Step 11: If choice is 6:

- **Display Savings Account details.**
- **Display Current Account details.**

Step 12: If choice is 7:

- Display "exiting".
- Stop the program.

Step 13: If an invalid choice is entered:

- Display "invalid choice".

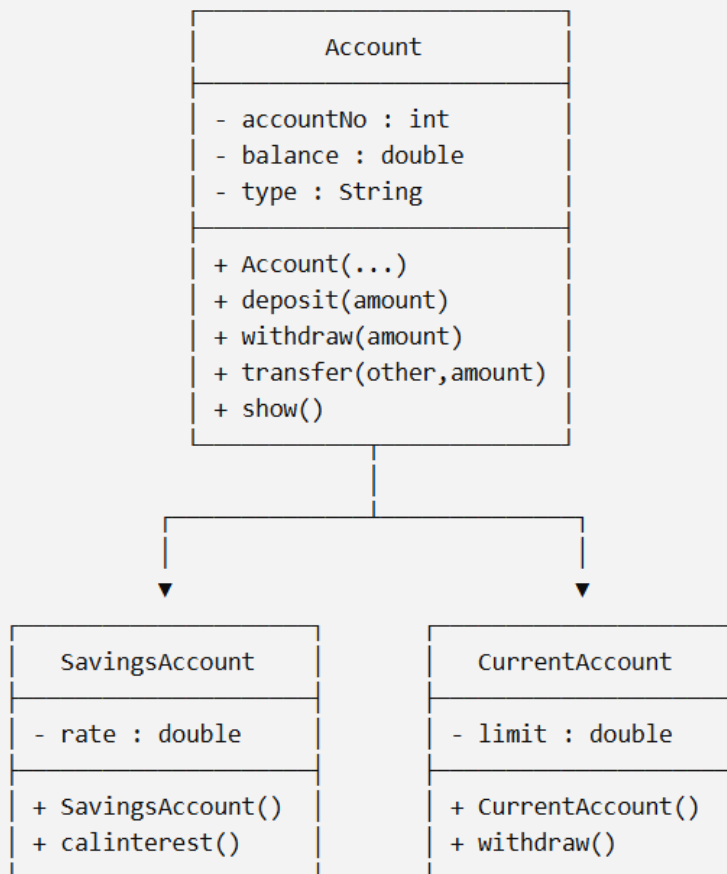
Step 14: Repeat Steps 4–13 until choice 7.

Step 15: Close Scanner.

Step 16: Stop.

3. UML Class Diagram

The main concept in your program is **inheritance**:



4.CODE:

```
import java.util.Scanner;
```

```
class Account{
```

```
int accountNo;  
double balance;  
String type;
```

```
Account(int num,double bal,String t){  
    accountNo=num;  
    balance=bal;  
    type=t;  
}
```

```
void deposit(double amount) {  
    balance+=amount;  
  
    System.out.println("amount you deposited is:"+amount);  
    System.out.println("total balance is:"+balance);  
}
```

```
void withdraw(double amount) {  
    if(amount<=balance) {  
        balance-=amount;  
        System.out.println("withdrawl amount:"+amount);  
        System.out.println("total balance remined:"+balance);  
    }else {  
        System.out.println("insufficient balance");  
    }  
}
```

```
void transfer(Account other,double amount) {  
    if(amount<=balance) {  
        balance-=amount;  
        other.balance+=amount;  
        System.out.println("transfer done succesfully..!");  
    }  
}
```

```

        System.out.println("updated balance:"+balance);
    }else {
        System.out.println("insufficient balance and transition failed..!");
    }
}

void show() {
    System.out.println("account number:"+accountNo);
    System.out.println("account type:"+type);
    System.out.println("total Balance:"+balance);
}
}

```

```

class SavingsAccount extends Account{
    double rate;

    SavingsAccount(int num,double bal,double r){
        super(num,bal,"savings account");
        rate=r;
    }

    void calinterest() {
        double interest=balance*rate/100;
        System.out.println("Interest:"+interest);
    }
}

```

```

class CurrentAccount extends Account{
    double limit;

    CurrentAccount(int num,double bal,double l){
        super(num,bal,"current account");
        limit=l;
    }
}

```

```
}
```

```
@Override
```

```
void withdraw(double amount) {
```

```
    if(amount<=balance+limit){
```

```
        balance-=amount;
```

```
        System.out.println("withdrawl is ok..,balance:"+balance);
```

```
    }else {
```

```
        System.out.println("overdraft limit crossed");
```

```
    }
```

```
}
```

```
}
```

```
//main
```

```
public class Week12 {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc=new Scanner(System.in);
```

```
        SavingsAccount sav=new SavingsAccount(100,5000,5);
```

```
        CurrentAccount cur=new CurrentAccount(101,3000,2000);
```

```
        int choice;
```

```
        do {
```

```
            System.out.println("\n--- Bank Menu ---");
```

```
            System.out.println("1. Deposit (Savings)");
```

```
            System.out.println("2. Withdraw (Savings)");
```

```
            System.out.println("3. Interest (Savings)");
```

```
            System.out.println("4. Withdraw (Current)");
```

```
            System.out.println("5. Transfer Savings -> Current");
```

```
            System.out.println("6. Show Accounts");
```

```
System.out.println("7. Exit");  
System.out.print("Choice: ");  
choice = sc.nextInt();
```

```
switch(choice) {
```

```
case 1:
```

```
    System.out.println("Amount:");  
    sav.deposit(sc.nextDouble());  
    break;
```

```
case 2:
```

```
    System.out.println("amount:");  
    sav.withdraw(sc.nextDouble());  
    break;
```

```
case 3:
```

```
    sav.calinterest();  
    break;
```

```
case 4:
```

```
    System.out.println("amount:");  
    cur.withdraw(sc.nextDouble());  
    break;
```

```
case 5:
```

```
    System.out.println("amount:");  
    sav.transfer(cur, sc.nextDouble());  
    break;
```

```
case 6:
```

```
    System.out.println("Savings:");  
    sav.show();  
    System.out.println("Current:");  
    cur.show();  
    break;
```

```
case 7:
```

```
    System.out.println("exiting...!");
```

```

        break;

    default:

        System.out.println("invalid choice..");

    }

}while(choice!=7);

    sc.close();

}

}

```

5.output:

```

:~\Users\Harsha Vardhan\AppData\Roaming\Code\User\workspaceStorage\f8641d91bba67b5feb3c9719e0895bbc\redhat.java\jdt_ws\java_61d938da\bin' 'Week
1'

--- Bank Menu ---
1. Deposit (Savings)
2. Withdraw (Savings)
3. Interest (Savings)
4. Withdraw (Current)
5. Transfer Savings -> Current
6. Show Accounts
7. Exit
Choice: 1
Amount:
200
amount you deposited is:200.0
total balance is:5200.0

--- Bank Menu ---
1. Deposit (Savings)
2. Withdraw (Savings)
3. Interest (Savings)
4. Withdraw (Current)
5. Transfer Savings -> Current
6. Show Accounts
7. Exit
Choice: 7
exiting...!
PS C:\Users\Harsha Vardhan\Desktop\java>

```